Project 2 – Acme Analytics Linux Platform

Day 8 · Phase 4: signals, lock, trap and a real queue system with a worker

Muhammad Bilal Javed · 25 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	stor01
Status	Phase 4 🟡 – Steps 4.1, 4.2 ✅ (Day 7) · Step 4.3 🟡 – concept ✅, archive queue system Steps 1–3 ✅ (queue + producer, worker, worker as a service); Steps 4–9 next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 7.

---

1. Step 4.3 – the concept: two copies, signals, lock and trap

💡 **In plain words:** jobs that run by themselves – nightly backups, exports – must never run **twice at the same time**, and when they are stopped they must not leave broken files behind. Three tools solve this: a **lock**, **signals** and **trap**.

Concept	Question it answers	Key line
Lock	Can a second copy start while the first is running?	`if [ -e "$LOCK" ]` · `flock -n` · a systemd service runs only once
Signal	How is a process stopped?	`kill -15` (SIGTERM) · Ctrl+C (SIGINT) · `kill -9` (SIGKILL)
trap	What is cleaned up when it is stopped?	`trap cleanup SIGINT SIGTERM`
`.part` file	How do I avoid a broken file that looks finished?	write to `file.part`, then `mv` when complete

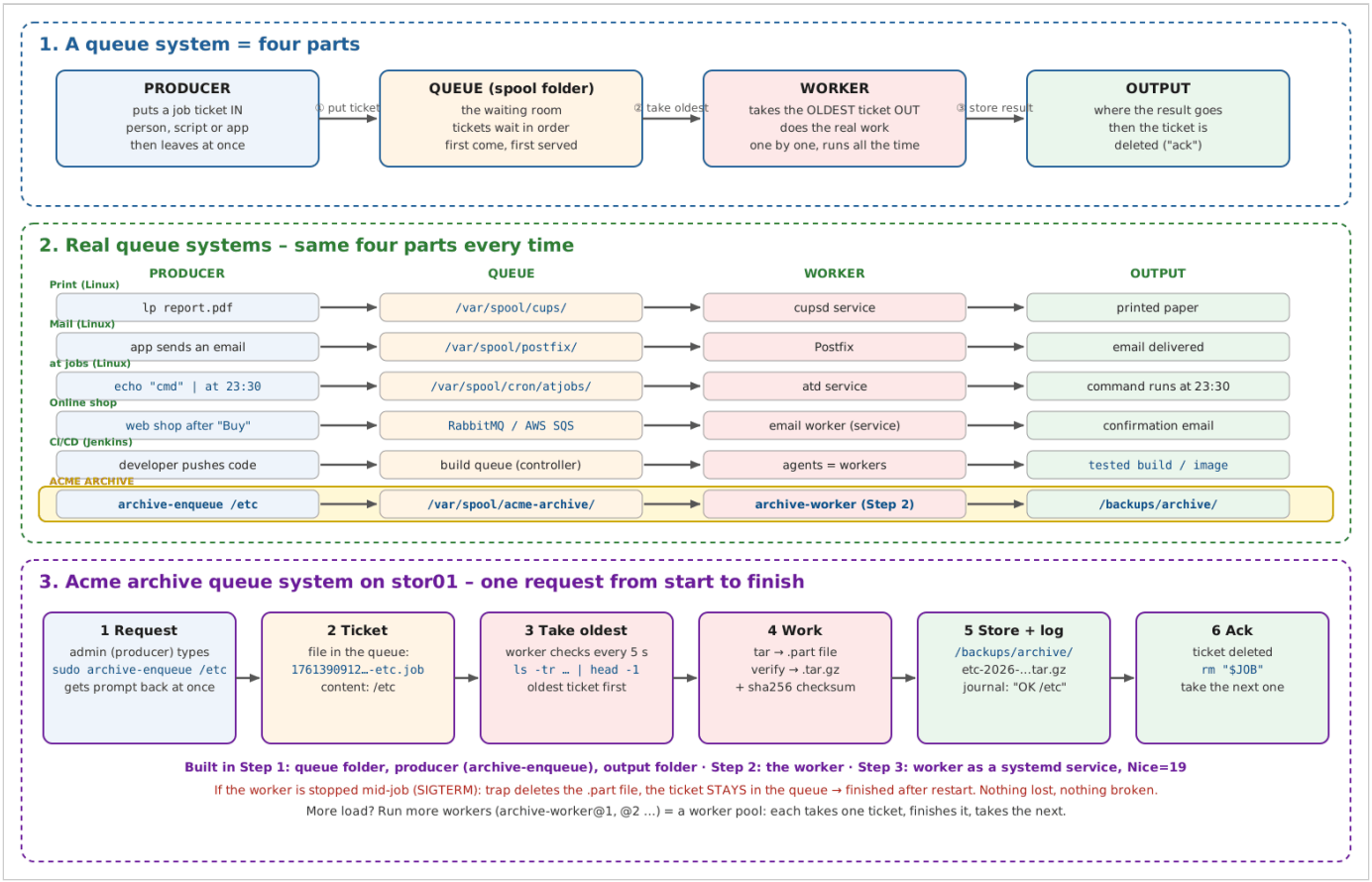
Key understanding (in my own words)

A nightly backup or export started by a **timer or cron** must never run **twice at the same time**: if the timer's backup runs from 02:00 to 03:00 and I start it by hand at 02:05, two copies would write to the same place and produce a broken result. The protection is a **lock** – a "someone is working on this right now" sign. Some programs have a lock **built in** (apt and dpkg – which is why a second apt, often Ubuntu's automatic updates running in the background, answers *"Could not get lock"*), but simple tools like `cp` or `rsync` and my own scripts have **none**, so I add it myself: a lock file checked at the start, `flock -n` (a kernel lock released automatically even after `kill -9`), or simply a systemd service, which never runs twice. A lock only stops a **second copy from starting**; it never stops anyone from terminating the job, and once the first copy has finished and removed its lock, the job can run again normally. A job is stopped by a **signal** from the kernel: **SIGTERM** (`kill -15`, "please stop"; `systemctl stop` sends it), **SIGINT** (Ctrl+C) or **SIGKILL** (`kill -9`, "stop now"). Stopping affects resources in two ways: **CPU and memory** are freed by the kernel whichever signal is used, but files the job created on **disk** – its lock, temporary files, a half-written archive – stay behind unless clean-up code removes them. **Linux processes have no clean-up by default**: the default action of SIGTERM is "die immediately", and SIGTERM only **gives the program a chance** to clean up. rsync, apt and databases have that code built in, while `cp`, `scp` and a plain Bash script do not – stopping a 20 GB `cp` halfway leaves a broken half-file that looks complete. **`trap`** adds clean-up code to a script: `trap cleanup SIGINT SIGTERM` means "on SIGTERM or Ctrl+C, first run my `cleanup` steps, then exit", and it runs only the steps I write. Trap does not stop or restart the process – the signal stops it, the trap cleans the disk before it ends, so the next start begins clean. **SIGKILL cannot be trapped**, so after `kill -9` the leftovers stay; a leftover lock is a **stale lock**, and the fix is always: prove nothing is running (`pgrep -af <job>`), remove the lock by hand, start the job again. Good jobs also write into a temporary **`.part` file** and rename it only when it is complete and verified, so a half-written file never looks finished.

2. Step 4.3 – the concept: queue systems, producers and workers

💡 **In plain words:** a **lock** says "busy, go away". A **queue with a worker** says "wait in line, I'll do it for you": everyone drops a request and leaves at once, and one program works through the requests one by one, at low priority, without losing any.

📸 **Architecture – a queue system has four parts; Linux already has several (print, mail, `at`), and the same pattern runs online shops and Jenkins:**



queue systems

Concept	Question it answers
Scheduler (timer, cron)	**When** does a fixed job start? – driven by time
Socket	Start a service **live** for each connection – the caller waits for the answer
Queue	In **what order** do waiting jobs run? – FIFO, first in, first out
Producer	Who **puts** a job ticket in the queue? – a person, a script or an application
Worker	Who **does** the jobs, one by one, any time? – a program that runs all the time
nice / ionice	How much **CPU / disk** may it use? – a different question from the order

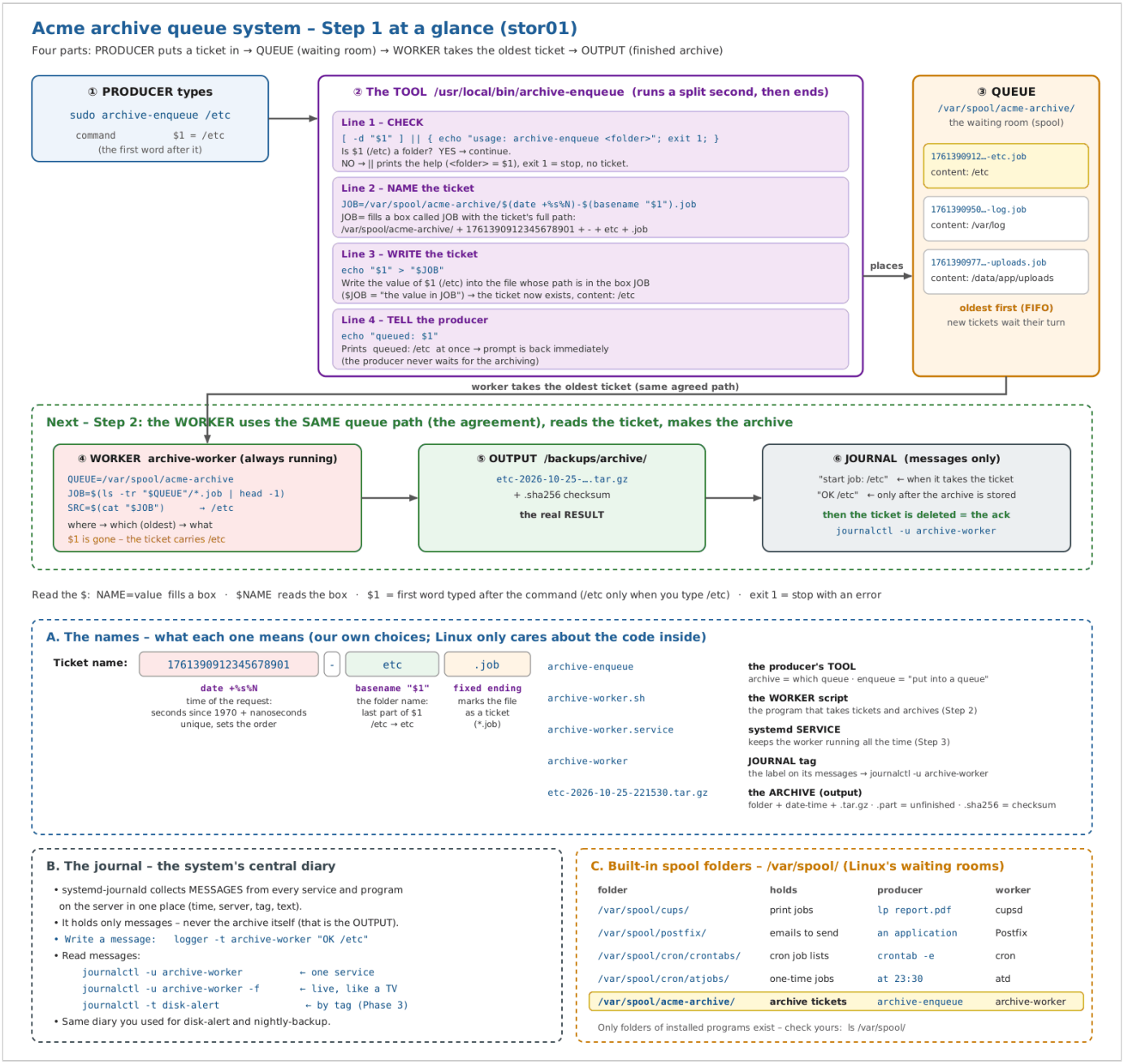
Key understanding (in my own words)

A **queue system** has four parts working together. A **producer** – a person, a script or an application – creates a **job ticket**, a small instruction such as *“archive /etc”*, places it in the **queue** and **gets control back at once**: it never waits for the job to be finished. The queue is a waiting room on disk called a **spool** folder; Linux keeps its own under **`/var/spool/`** – print jobs from `lp` wait in `/var/spool/cups/` for the worker `cupsd`, emails wait in `/var/spool/postfix/` for Postfix, `at` jobs wait in `/var/spool/cron/atjobs/` for `atd`. A **worker**, a program that runs all the time (usually a systemd service), takes the tickets **in order of arrival** – the oldest first, **FIFO**, first in, first out – does the work, stores the result in the agreed **output** place, and only then deletes the ticket (the **ack**, “done”), so if it fails or is stopped halfway the ticket stays and nothing is lost. Linux does not build a queue by itself: someone must place the tickets, and the worker must be written to take them out. Developers write workers; admins run and manage them – a systemd service with `Restart=on-failure`, more copies (a **worker pool**) when the load grows, logs in the journal, a safe stop with SIGTERM. Workers exist for three reasons: **nobody waits**, **the load stays under control**, **nothing is lost**. The same pattern runs a web shop (the shop puts *“send the confirmation email for order 501”* into RabbitMQ or AWS SQS and an email worker sends it) and **Jenkins** (the controller keeps the build queue, *“Waiting for next available executor”*, and agents are the workers). A **timer** is driven by **time** – one fixed job at a fixed time; a **socket** answers each **live connection** at once while the caller waits; a **worker** is driven by **stored requests** – any job, any time, one by one, while the requester has already left. Real automation grows step by step: do it by hand, script it, schedule it, and finally let a worker do it for everyone – repeating the same proven steps like a tireless user, without deciding anything new as an AI agent would. **`nice`** answers a different question from the queue: the queue decides the **order**, nice (-20 most important … 19 politest) decides **how much CPU** a running job gets.

3. Lab – the Acme archive queue system on stor01

💡 **The story:** at Acme, admins and scripts request archives (“archive /etc before I change the config”, “archive the uploads”). Instead of running heavy `tar` jobs by hand, they drop a **ticket** into a queue, and one **archive worker** turns the tickets into verified archives one by one, at low priority, so the customer application is never slowed down.

📸 **Architecture – one request from start to finish, the names, the journal and the built-in spool folders:**



step 1 overview

Part	Path
Producer’s tool	`/usr/local/bin/archive-enqueue`
Queue (spool folder)	`/var/spool/acme-archive/` – tickets like `1761390912345678901-etc.job`, content `/etc`
Worker	`/usr/local/bin/archive-worker.sh`, run as `archive-worker.service`
Output	`/backups/archive/` – `etc-2026-10-25-234228.tar.gz` + `.sha256`
Messages	the journal – `journalctl -u archive-worker`

3.1 Step 1 – the queue and the producer

```
# stor01
sudo mkdir -p /var/spool/acme-archive /backups/archive
sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }
JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
sudo chmod +x /usr/local/bin/archive-enqueue
```

Line	Meaning
`[ -d "$1" ] || { echo "usage: …"; exit 1; }`	$1 = the first word typed after the command (the folder); `-d` asks "is it a folder?"; if not, `||` prints the help (`<folder>` describes $1) and `exit 1` stops – no ticket
`JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job`	JOB= fills a box with the ticket's path: queue folder + time of the request (seconds since 1970 + nanoseconds – unique, sets the order) + folder name (`basename` keeps the last part) + `.job`
`echo "$1" > "$JOB"`	Write the folder path into the ticket file ($JOB = the value in the box) – the instruction for the worker
`echo "queued: $1"`	Tell the producer at once – the prompt is back immediately

📸 **Evidence:**

```
bilal_003@stor01:~$ sudo mkdir -p /var/spool/acme-archive /backups/archive
bilal_003@stor01:~$ sudo -v
bilal_003@stor01:~$ sudo tee /usr/local/bin/archive-enqueue > /dev/null <<'EOF'
#!/bin/bash
[ -d "$1" ] || { echo "usage: archive-enqueue <folder>"; exit 1; }

JOB=/var/spool/acme-archive/$(date +%s%N)-$(basename "$1").job
echo "$1" > "$JOB"
echo "queued: $1"
EOF
bilal_003@stor01:~$ sudo bash -n /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ sudo chmod +x /usr/local/bin/archive-enqueue
bilal_003@stor01:~$ _
```

step 1

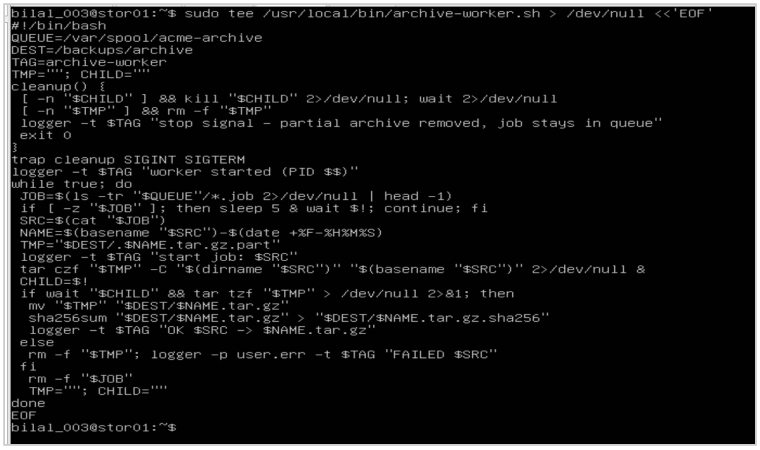
Lesson: my first version had `data` instead of `date`. `bash -n` did not catch it – it checks only the grammar of a script, not whether its commands exist. Only a test run proves a script works.

3.2 Step 2 – the worker

The full script is in the screenshot below. The lines that carry the concept:

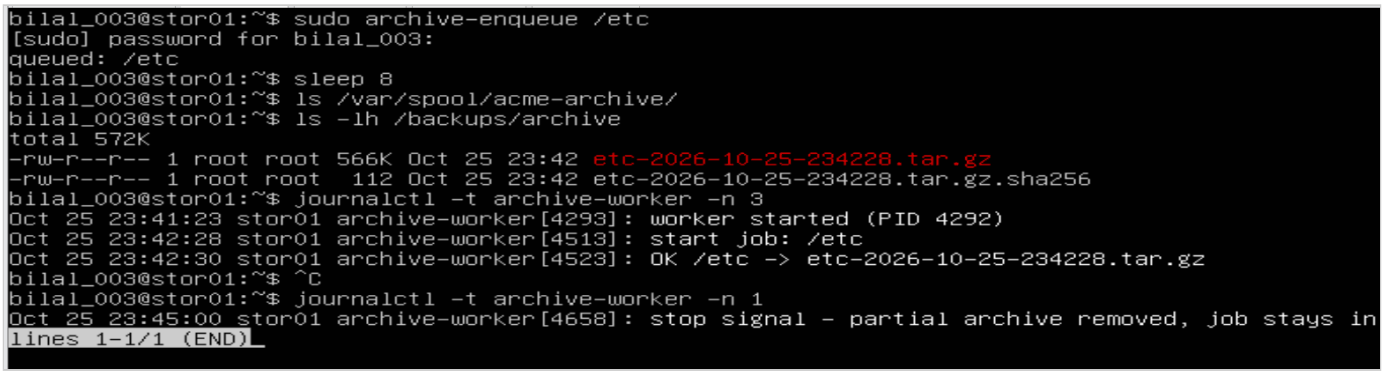
```
QUEUE=/var/spool/acme-archive                          # WHERE – the agreed queue folder (same path as the producer)
DEST=/backups/archive                                  # OUTPUT
trap cleanup SIGINT SIGTERM                            # TRAP – on SIGTERM/Ctrl+C: stop tar, delete the .part file
while true; do                                         # WORKER – runs forever
  JOB=$(ls -tr "$QUEUE"/*.job 2>/dev/null | head -1)   # WHICH – the oldest ticket first (FIFO)
  SRC=$(cat "$JOB")                                    # WHAT – read the folder from the ticket
  TMP="$DEST/.$NAME.tar.gz.part"                       # SAFE FILE – work into a .part file
  logger -t $TAG "start job: $SRC"                     # JOURNAL – message only
  mv "$TMP" "$DEST/$NAME.tar.gz"                       # OUTPUT – only a complete, verified archive gets the real name
  logger -t $TAG "OK $SRC -> $NAME.tar.gz"             # JOURNAL – "OK" only after the archive is stored
  rm -f "$JOB"                                         # ACK – delete the ticket
done
```

📸 **Evidence – the worker script:**



worker script

📸 Evidence – manual test: worker running in one terminal; in the other `queued: /etc` at once, the queue empty again (ack), `etc-2026-10-25-234228.tar.gz` (566 K) + `.sha256` in the output, the journal *worker started* → *start job* → *OK* within 2 seconds; Ctrl+C → the trap's *"stop signal"* message:



enqueue archive journal trap

Lessons: I first typed the worker by hand with five mistakes – `backups/archive` without the leading `/`, `[ -n "$CHILD"]` without the space before `]`, `SIGNIT`, `sha56sum` and, worst, **`*.JOB` instead of `*.job`**: Linux is case-sensitive, so the worker would have waited forever without any error. Long scripts are pasted, not retyped. The number in `archive-worker[4513]` is the PID of each `logger` call; the worker's own PID is the one in the message (4292).

3.3 Step 3 – the worker as a service, at low priority

```
sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
sudo systemctl daemon-reload
sudo systemctl enable --now archive-worker
systemctl status archive-worker --no-pager
ps -o pid,ni,cmd -C archive-worker.sh
```

Setting	Meaning
`ExecStart=`	The worker program
`Restart=on-failure`	Restart it automatically if it crashes
`Nice=19` / `IOSchedulingClass=idle`	Lowest CPU priority / use the disk only when nobody else needs it (= `nice -n 19`, `ionice -c3`)
`WantedBy=multi-user.target` + `enable`	Start at every boot – a link in `multi-user.target.wants/`

📸 Evidence – the unit; `verify` finds nothing in my file (the yellow line is Ubuntu's own `snapd.service`):

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/archive-worker.service > /dev/null <<'EOF'
[Unit]
Description=Acme archive worker (processes the archive queue)
After=local-fs.target
[Service]
ExecStart=/usr/local/bin/archive-worker.sh
Restart=on-failure
Nice=19
IOSchedulingClass=idle
[Install]
WantedBy=multi-user.target
EOF
bilal_003@stor01:~$ sudo systemd-analyze verify /etc/systemd/system/archive-worker.service
/lib/systemd/system/snapd.service:23: Unknown key name 'RestartMode' in section 'Service', ignoring
bilal_003@stor01:~$ _
```

service unit

📸 **Evidence – `active (running)`, Main PID 4839, 1.6 MB; the cgroup holds the worker and its child `sleep 5` (empty queue, waiting); NI 19:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now archive-worker
Created symlink /etc/systemd/system/multi-user.target.wants/archive-worker.service → /etc/systemd/sy
stem/archive-worker.service.
bilal_003@stor01:~$ systemctl status archive-worker
● archive-worker.service - Acme archive worker (processes the archive queue)
     Loaded: loaded (/etc/systemd/system/archive-worker.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-10-25 23:53:46 UTC; 22s ago
   Main PID: 4839 (archive-worker.)
      Tasks: 2 (limit: 964)
     Memory: 1.6M
        CPU: 130ms
     CGroup: /system.slice/archive-worker.service
             ├─4839 /bin/bash /usr/local/bin/archive-worker.sh
             └─4860 sleep 5

Oct 25 23:53:46 stor01 systemd[1]: Started Acme archive worker (processes the archive queue).
Oct 25 23:53:46 stor01 archive-worker[4840]: worker started (PID 4839)
bilal_003@stor01:~$ ps -o pid,ni,cmd -C archive-worker.sh
    PID  NI CMD
   4839  19 /bin/bash /usr/local/bin/archive-worker.sh
bilal_003@stor01:~$
```

service running

Key understanding – the lab so far (in my own words)

> On stor01 I built the **Acme archive queue system**. In **Step 1** I created the **queue** `/var/spool/acme-archive/` (the waiting room for job tickets), the **output** `/backups/archive/` and the **producer's tool** `archive-enqueue` in `/usr/local/bin`, the folder for an admin's own commands. When I type `sudo archive-enqueue /etc`, `$1` – a box filled with the first word after the command – becomes `/etc`; `[ -d "$1" ]` checks it is a folder (otherwise `||` prints the help and `exit 1` stops); `JOB=` builds the ticket's path from the queue folder, the time of the request (`date +%s%N`) and the folder name (`basename`); `echo "$1" > "$JOB"` writes the folder into that ticket file, and *"queued: /etc"* comes back at once. The tool's name is only a label for humans – "enqueue" is the standard word for "put into a queue" – and what it does is decided only by its code. In **Step 2** I created the **worker** `archive-worker.sh`. It knows where to look because the **same queue path** is written at its top – producer and worker simply agree on it, as `cupsd` reads its spool folder from its settings file – and it reads the folder from the ticket with `cat`, because `$1` exists only for the split second the producer runs. In a manual test it turned my ticket into `etc-2026-10-25-234228.tar.gz` with a `.sha256` checksum within 2 seconds, wrote *start job* and *OK* to the **journal** (the server's central diary, which holds only messages, never the archive itself), deleted the ticket (the ack), and on Ctrl+C its trap logged *"stop signal"* before exiting. In **Step 3** I made it a **systemd service** with `Restart=on-failure`, `Nice=19` and `IOSchedulingClass=idle`; it now runs all the time as PID 4839 at **NI 19**, its cgroup showing it waiting (`sleep 5`) for the next ticket. Because a service runs only once, systemd itself is the lock – no second worker can start.

4. Next steps

1. Step 4.3 Steps 4–9: three jobs in the queue · a request at any time (worker vs timer) · stop the worker in the middle of a job (trap test) · a deliberate **L1 → L2 escalation** (`/backups` full) and **L2 → L3** (a GLPI Problem left Pending) · clean-up.
2. Step 4.4 zombie processes (optional) · Step 4.5 cgroups (`MemoryMax`, `CPUQuota`) · the first incident drill · VMware snapshot `50-processes`.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation's *Connect to Server*.